

ONE HEALTH PANDEMIC AND DISEASE OUTBREAK FORECASTING

A Practical Technical and AI-Assisted Modelling Workbook — Phase 2

Google Colab · Python · Gemini

Table of Contents

How to Use This Workbook.....	2
MODULE 1 — Introduction to One Health and Pandemic Forecasting	2
MODULE 2 — Framing the Epidemiological Problem	4
MODULE 3 — The Google Colab Environment.....	5
MODULE 4 — Understanding and Ingesting the Dataset.....	6
MODULE 5 — Data Cleaning and Preprocessing.....	8
MODULE 6 — Exploratory Data Analysis.....	9
MODULE 7 — Time-Series Analysis Fundamentals.....	10
MODULE 8 — Baseline Forecasting Models	12
MODULE 9 — Chronological Train / Validation / Test Design.....	12
MODULE 10 — Feature Engineering.....	14
MODULE 11 — Model Development	15
MODULE 12 — Model Training and Troubleshooting.....	16
MODULE 13 — Model Evaluation	17
MODULE 14 — Uncertainty and Forecast Intervals	17
MODULE 15 — Outbreak Detection and Early Warning.....	18
MODULE 16 — Interpreting the Model.....	19
MODULE 17 — Gemini as an AI Assistant: Scope and Limits.....	20
MODULE 18 — Prompt Engineering for Modelling Tasks	20
MODULE 19 — Data Privacy, Ethics, and Responsible AI Use	21
MODULE 20 — Capstone Project	22
APPENDIX A — AI-Assistance Log Template.....	23
APPENDIX B — Troubleshooting Guide	23
APPENDIX C — Glossary	25
APPENDIX D — References and Data Sources.....	26

How to Use This Workbook

This workbook trains public health, epidemiology, surveillance, and One Health professionals to build a real disease-outbreak forecasting model — first by hand, then with Gemini as an assistant inside Google Colab.

The guiding rule for every module is the same:

UNDERSTAND → IMPLEMENT → VALIDATE → USE AI TO ASSIST → CRITICALLY EVALUATE → IMPROVE → INTERPRET

You will not be shown how to ask an AI to “build a forecasting model” on your behalf. You will build one yourself, in Python, in Google Colab, understanding every step — and then learn where Gemini can genuinely save you time, and where it must never be trusted to decide.

By the end, you will be able to explain and defend a forecasting model scientifically: why it was built the way it was, what data it used, how it was evaluated, what its numbers mean, and — just as important — what it cannot tell you.

Dataset used throughout this workbook: WHO COVID-19 weekly case data via Our World in Data (<https://covid.ourworldindata.org/data/owid-covid-data.csv>), which aggregates official WHO and national surveillance reports. It is public, non-identifiable, and freely reusable — precisely the kind of dataset you should use when practicing with an AI assistant such as Gemini. No patient-level or identifiable data appears anywhere in this workbook.

MODULE 1 — Introduction to One Health and Pandemic Forecasting

Concept

One Health recognizes that human, animal, and environmental health are interconnected — most emerging infectious diseases (Ebola, avian influenza, COVID-19) originate at that interface. **Surveillance** is the continuous collection and analysis of health data to detect abnormal patterns. An **outbreak** is a localized increase in cases above what is expected; an **epidemic** is a wider increase across a region; a **pandemic** is a global epidemic. **Forecasting** estimates future case counts from historical patterns; **early warning** flags when observed activity deviates enough from the expected baseline to warrant investigation.

Why Modelling Matters

Surveillance teams generate large volumes of case data every week. A model turns that raw history into a structured, testable statement about the near future — informing staffing, stockpiling, and response timing. It is a **decision-support tool**, not a crystal ball.

The One Health Foundation

Figure 1 shows human, animal, and environmental surveillance converge into shared One Health data.

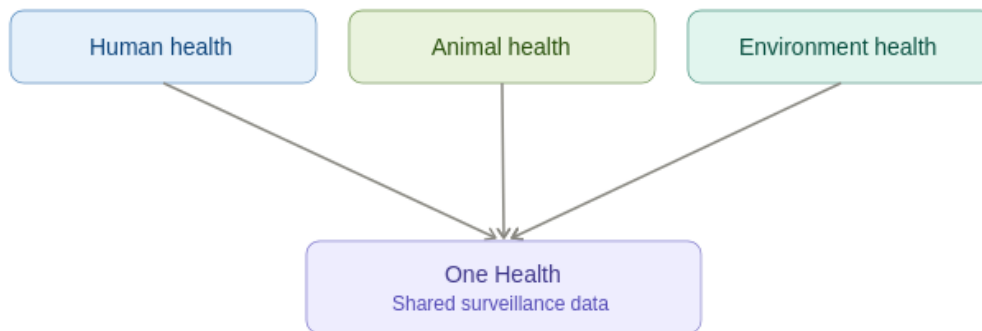


Figure 1: One Health convergence

The Data → Decision Pipeline

Figure 2 shows the core pipeline every module in this workbook builds toward.



Figure 2: Data to decision pipeline

Data → Analysis → Model → Forecast → Decision Support

Each arrow is a place where error, bias, or uncertainty can enter. A forecasting workbook must teach you to recognize that at every stage.

Limitations You Must Internalize Now

- A model reflects the data it was trained on — reporting delays, testing changes, and under-reporting all bias the “truth” it has learned.
- Forecasts degrade the further into the future they reach.

- A forecast is a statement of *probability*, never certainty.
- No model can substitute for field epidemiological investigation.

CHECK YOUR UNDERSTANDING

1. Why is a pandemic forecast fundamentally different from a weather forecast in terms of the *consequences* of being wrong?
2. Name two ways under-reporting could bias a forecasting model trained on official case counts.

KEY TAKEAWAY

Forecasting models are decision-support tools built from imperfect surveillance data. Every module from here builds toward a model you can explain, defend, and know the limits of.

MODULE 2 — Framing the Epidemiological Problem

Before writing a line of code, define the modelling problem in plain language, then formally.

Example problem statement: *“Can we forecast weekly reported COVID-19 cases in a given country over the next 4 weeks, using that country’s own historical case history?”*

Element	Definition for our scenario
Target variable	new_cases (weekly reported cases)
Predictors	lagged case counts, rolling averages, calendar features
Prediction horizon	4 weeks ahead
Unit of analysis	country-week
Geographic scope	one country at a time (start simple)
Temporal resolution	weekly (aggregated from daily reports)
Success criterion	forecast error meaningfully lower than a naive baseline

HANDS-ON EXERCISE

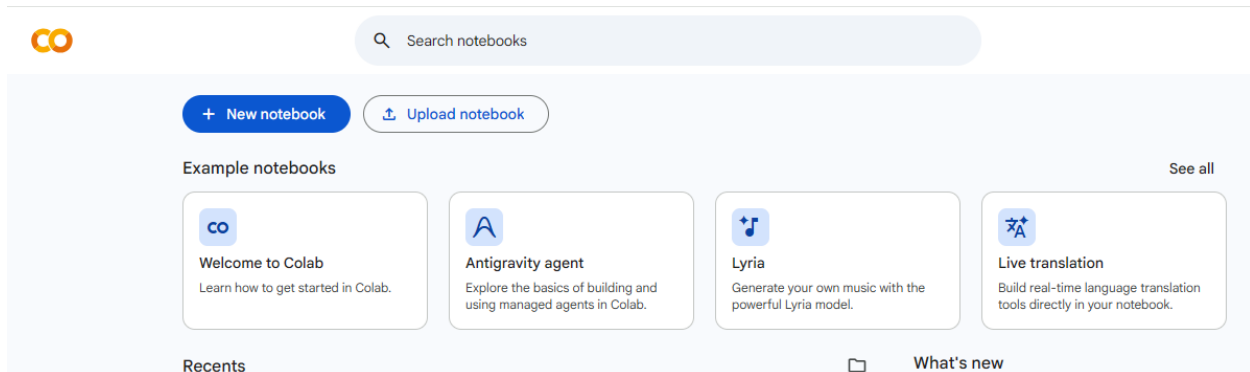
Write your own one-sentence problem statement for a surveillance question relevant to your own work (e.g., cholera cases in a district, avian influenza detections in poultry). Fill in the same table for it. You will reuse this framing in the capstone.

KEY TAKEAWAY

A vague goal (“predict outbreaks”) produces a vague, unusable model. A precise target, horizon, and unit of analysis is what actually makes a model buildable and testable.

MODULE 3 — The Google Colab Environment

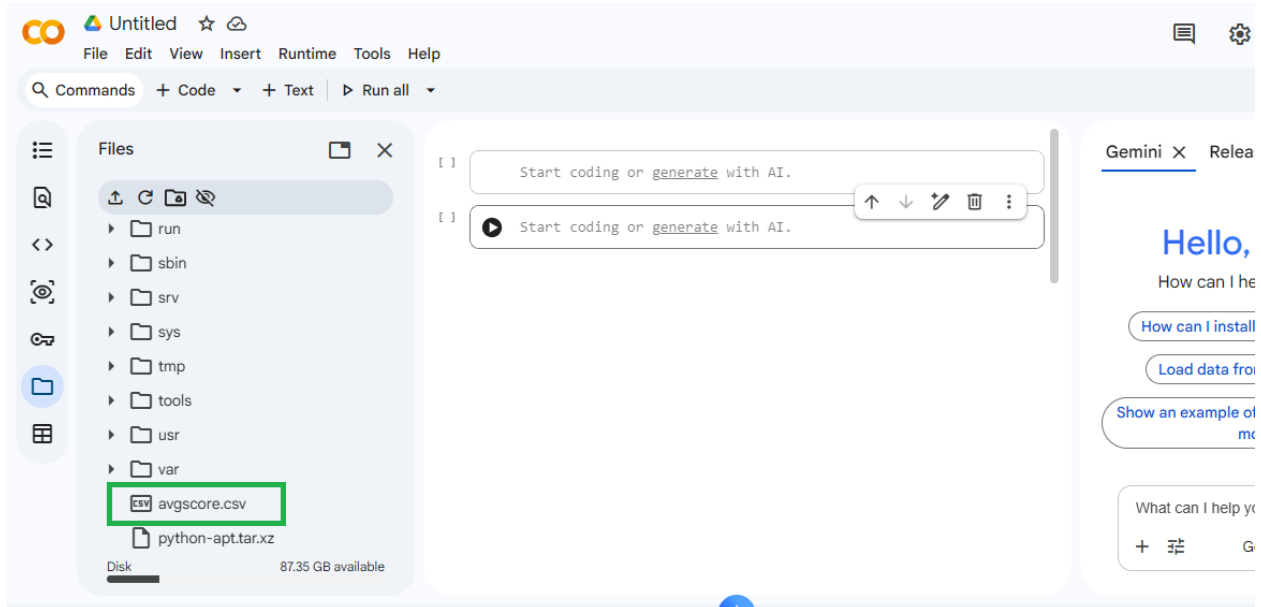
Google Colab is a free, browser-based Jupyter notebook environment with no installation required.



Google Colab landing page with “New Notebook” button

Core Concepts

- **Notebook:** a .ipynb file made of cells, saved automatically to Google Drive.
- **Code cell:** runs Python; press Shift+Enter to execute.
- **Markdown cell:** for notes, headings, and explanations (like this document).
- **Runtime:** the virtual machine executing your code. Runtime → Restart runtime clears all variables — a common fix when things “look stuck.”
- **Installing packages:** !pip install package_name (most data-science packages are pre-installed).
- **Uploading data:** `from google.colab import files; files.upload()`, or mount Drive with `from google.colab import drive; drive.mount('/content/drive')`.



Files panel showing an uploaded CSV in the Colab runtime

Good Practice

- Name your notebook meaningfully: `OneHealth_Forecasting_Phase2.ipynb`.
- Use markdown cells as section headers matching this workbook's modules.
- Save a copy to Drive (File → Save a copy in Drive) before major changes.
- Never assume your data persists — the runtime disk resets when the session disconnects; Drive-mounted files or re-uploaded files survive, in-memory variables do not.

GEMINI ASSISTANT PROMPT

"I'm new to Google Colab. Explain, in plain language, the difference between a code cell and a markdown cell, what happens when I restart the runtime, and why my uploaded CSV might disappear after a few hours of inactivity."

What Gemini is being asked to do: explain environment mechanics, not write analysis code. **What to check:** the explanation should mention session timeouts/disconnection — if it claims uploaded files persist indefinitely without Drive, that's wrong; verify against Colab's own documentation.

KEY TAKEAWAY

Colab removes the setup barrier so you can focus on the science. Understand its session model now — it prevents "my data disappeared" panic later.

MODULE 4 — Understanding and Ingesting the Dataset

Data Dictionary (subset used in this workbook)

Column	Type	Meaning
location	categorical	country name
date	date	daily report date
new_cases	numeric	newly reported cases that day
new_deaths	numeric	newly reported deaths that day
population	numeric	country population (for rate calculations)

Source: Our World in Data COVID-19 dataset, compiled from WHO and national health-authority reports — <https://covid.ourworldindata.org/data/owid-covid-data.csv> (accessed for this workbook: 2026).

Traditional Technical Approach

```
import pandas as pd
```

```

url = "https://covid.ourworldindata.org/data/owid-covid-data.csv"
df = pd.read_csv(url, usecols=["location", "date", "new_cases", "new_deaths",
"population"])

df["date"] = pd.to_datetime(df["date"])

country = "South Africa"          # change to the country you are studying
df = df[df["location"] == country].sort_values("date").reset_index(drop=True)

print(df.shape)
print(df.dtypes)
print(df.head())
print(df.isna().sum())
print(df.duplicated(subset="date").sum())

```

CODE WALKTHROUGH

- `usecols` limits memory use to the columns we actually need.
- `pd.to_datetime` converts the date column from text to a real date type — essential for any time-series operation later.
- Filtering to one location keeps the scenario at the “one country” unit of analysis defined in Module 2.
- `isna().sum()` and `duplicated()` are your first data-quality check, before any cleaning decisions are made.

EXPECTED OUTPUT — WILL VARY DEPENDING ON DATA

A DataFrame with several hundred to over a thousand rows (daily records for the chosen country), some missing values in `new_cases/new_deaths` (especially early and late in the series), and typically zero duplicate dates.

GEMINI-ASSISTED METHOD

GEMINI ASSISTANT PROMPT

“I have loaded a pandas DataFrame called `df` containing daily COVID-19 surveillance data for one country, with columns `location`, `date`, `new_cases`, `new_deaths`, `population`. Generate Python code to display the shape, column data types, count of missing values per column, count of duplicate dates, and summary statistics for the numeric columns. Do not modify the DataFrame. Explain what each line does.”

What Gemini is being asked to do: generate an *inspection* script, not clean or alter data. **What to check:** confirm it does not silently drop rows, fill NaNs, or reassign `df` — inspection code should never mutate the data. **Common Gemini mistakes:** using `df.fillna(0)` “to make the summary cleaner” — this destroys the missing-data signal before you’ve decided how to handle it. **How to validate:** run the traditional code above and Gemini’s code side-by-side; the numeric outputs (shape, NaN counts) must match exactly.

GEMINI NOT TO BE TRUSTED FOR

Deciding *why* values are missing or whether a duplicate is a data error versus a legitimate re-report. That judgment stays with you.

HANDS-ON EXERCISE

Repeat the inspection for a second country and compare missingness patterns. Note in your AI-assistance log (Appendix A) whether you used Gemini's code or your own.

KEY TAKEAWAY

Before any modelling, you must know exactly what is in your data — its shape, types, and gaps — using code you can read and trust.

MODULE 5 — Data Cleaning and Preprocessing

Concept

Cleaning is not “make the data look tidy” — every operation must have a scientific justification, because every operation can also destroy a real epidemiological signal.

Critical warning: An unusually high case count may be a data-entry error — or it may be a genuine outbreak spike. Never delete an outlier without investigating which one it is.

Traditional Technical Approach

1. Handle negative case counts (common: retrospective corrections reported as negative deltas)

```
df["new_cases"] = df["new_cases"].clip(lower=0)
```

2. Resample to weekly totals – smooths day-of-week reporting artifacts

```
weekly = (  
    df.set_index("date")["new_cases"]  
        .resample("W")  
        .sum()  
        .reset_index()  
        .rename(columns={"new_cases": "weekly_cases"})  
)
```

3. Interpolate short internal gaps only (do NOT interpolate large gaps – flag them instead)

```
weekly["weekly_cases"] = weekly["weekly_cases"].interpolate(limit=2,  
limit_area="inside")
```

4. Flag remaining missing weeks rather than silently filling them

```
weekly["is_imputed_gap"] = weekly["weekly_cases"].isna()
```

CODE WALKTHROUGH

- `.clip(lower=0)` addresses a known reporting artifact (negative daily deltas from corrections) without touching genuine positive spikes.
- Weekly resampling matches the temporal resolution defined in Module 2 and reduces day-of-week noise (e.g., weekend under-reporting).
- `interpolate(limit=2, ...)` only fills small gaps (≤ 2 weeks); larger gaps are left as missing and explicitly flagged in `is_imputed_gap`, so nothing is silently invented.

GEMINI ASSISTANT PROMPT

“I have a weekly pandas Series called `weekly_cases` from disease surveillance data. Some values are missing. Write Python code that interpolates only gaps of 2 weeks or fewer, leaves longer gaps as NaN, and creates a boolean column flagging which values were imputed. Explain the reasoning and do not fill long gaps.”

Common Gemini mistakes: defaulting to `fillna(method='ffill')` or filling every gap regardless of length — this can mask a real reporting collapse (e.g., a surveillance system outage) as if cases genuinely dropped to nothing. **How to validate:** plot `weekly_cases` before and after, and manually confirm no large gap was silently filled.

GEMINI SHOULD NOT DECIDE

Whether a spike is a data error or a genuine outbreak — that requires epidemiological judgment (cross-checking with hospitalization/death data, news of an event, lab-confirmation records), not a statistical rule alone.

THINK LIKE AN EPIDEMIOLOGIST

A four-week gap suddenly appears in the middle of your series. List two real-world causes unrelated to disease activity that could explain it, and how you’d investigate before deciding to interpolate.

KEY TAKEAWAY

Every cleaning decision is a scientific claim about your data. Document it, and never let interpolation or outlier removal erase a real signal.

MODULE 6 — Exploratory Data Analysis

Traditional Technical Approach

```
import matplotlib.pyplot as plt
```

```
fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(weekly["date"], weekly["weekly_cases"], label="Weekly cases")
ax.plot(weekly["date"], weekly["weekly_cases"].rolling(4).mean(), label="4-
week rolling mean")
```

```
ax.set_title(f"Weekly reported cases – {country}")
ax.set_xlabel("Date"); ax.set_ylabel("Cases"); ax.legend()
plt.show()
```

INTERPRETATION QUESTIONS

- Does the rolling mean reveal a trend the raw series hides?
- Are there sharp peaks that could reflect either a genuine wave or a reporting artifact (e.g., a backlog cleared in one week)?
- Do peaks recur at similar times of year, suggesting seasonality?

GEMINI ASSISTANT PROMPT

“I have a pandas DataFrame `weekly` with columns `date` and `weekly_cases`. Generate matplotlib code to plot the raw series with a 4-week rolling average overlay, and separately a plot of week-over-week percentage change. Label axes and title clearly. Explain what patterns each plot is designed to reveal.”

What to check: Gemini should not add interpretive claims like “this shows an outbreak in March” — a plotting prompt should return plotting code and technique explanations, not epidemiological conclusions it cannot verify from a description alone.

HANDS-ON EXERCISE

Produce both plots for your chosen country, then write two sentences interpreting what you see — in your own words, not Gemini’s.

KEY TAKEAWAY

EDA turns numbers into a visual story that guides every decision from here forward — but the *interpretation* is your responsibility, not the plotting library’s.

MODULE 7 — Time-Series Analysis Fundamentals

Concept (plain language first)

A time series has three ingredients: a **trend** (the slow overall direction), **seasonality** (a pattern that repeats at fixed intervals), and **noise/residuals** (what’s left over). **Stationarity** means a series’ statistical properties (mean, variance) don’t change over time — many forecasting methods assume it, so we test for it and, if needed, transform the data (commonly by differencing) before modelling.

Autocorrelation measures how much a series is related to its own past values (ACF); **partial autocorrelation** (PACF) measures that relationship after removing the effect of shorter lags — both help choose which lagged values are worth using as model features.

Traditional Technical Approach

```
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

series = weekly.set_index("date")["weekly_cases"].dropna()

decomposition = seasonal_decompose(series, model="additive", period=52)
decomposition.plot()
plt.show()

adf_result = adfuller(series)
print(f"ADF statistic: {adf_result[0]:.3f}, p-value: {adf_result[1]:.3f}")
# p-value < 0.05 -> reject non-stationarity (series likely stationary)

plot_acf(series, lags=20)
plot_pacf(series, lags=20)
plt.show()
```

CODE WALKTHROUGH

- `seasonal_decompose` splits the series into trend, seasonal, and residual components (period=52 assumes yearly seasonality in weekly data).
- The Augmented Dickey-Fuller (`adfuller`) test formally checks stationarity; a low p-value supports treating the series as stationary.
- ACF/PACF plots guide which lag features (Module 10) are likely to carry predictive information.

GEMINI ASSISTANT PROMPT

“Explain, without heavy mathematical notation, what an Augmented Dickey-Fuller test result means for a public health analyst who has never studied time-series statistics. Then explain how to read an ACF plot to decide which lag features to use in a forecasting model.”

What to check: the explanation should stay conceptual and correct — flag it if Gemini claims stationarity means “no trend at all” (imprecise) rather than “statistical properties are stable over time.”

KEY TAKEAWAY

You don’t need to be a statistician to use these tools correctly — but you do need to know what question each one answers before trusting its output.

MODULE 8 — Baseline Forecasting Models

A sophisticated model is not useful if it cannot beat a simple baseline. Always build one first.

```
import numpy as np

s = weekly["weekly_cases"]

naive_forecast = s.shift(1)                # "next week = this week"
seasonal_naive_forecast = s.shift(52)     # "next week = same week
last year"
moving_avg_forecast = s.rolling(4).mean().shift(1) # "next week = avg of
last 4 weeks"

baseline_df = pd.DataFrame({
    "actual": s,
    "naive": naive_forecast,
    "seasonal_naive": seasonal_naive_forecast,
    "moving_avg": moving_avg_forecast,
})
```

GEMINI ASSISTANT PROMPT

"I have a weekly cases Series. Generate Python code for three baseline forecasts: naive (last observed value), seasonal naive (same week one year prior), and a 4-week moving average — each properly shifted so no future information leaks into the prediction for a given week. Explain why the shift is necessary."

Common Gemini mistake: forgetting `.shift(1)`, which would let the "forecast" for a week include that same week's actual value — a subtle but serious form of data leakage. **How to validate:** manually check one row — the naive forecast for week t must equal the actual value from week $t-1$, not week t .

KEY TAKEAWAY

Baselines set the bar. If your machine-learning model can't beat "just use last week's number," it isn't adding value yet.

MODULE 9 — Chronological Train / Validation / Test Design

Why Random Splitting Fails Here

Random train/test splitting assumes rows are independent. In a time series, a row is generated in part by the rows before it — randomly splitting lets the model "see the future" during training (temporal leakage / look-ahead bias), producing evaluation numbers that look great and are meaningless.

Chronological Split

2018 → 2019 → 2020 → 2021		2022		2023
TRAINING		VALIDATION		TEST

Figure 3 — the same split drawn as a timeline: nothing in Validation or Test is ever used to inform Training.



Figure 3: Chronological train validation test split

```
train = weekly[weekly["date"] < "2022-01-01"]
val    = weekly[(weekly["date"] >= "2022-01-01") & (weekly["date"] < "2023-01-01")]
test   = weekly[weekly["date"] >= "2023-01-01"]
```

For more robust evaluation, use **walk-forward (rolling-origin) validation**: repeatedly train on data up to time t , forecast the next horizon, then move t forward and repeat.

```
from sklearn.model_selection import TimeSeriesSplit

tscv = TimeSeriesSplit(n_splits=5)
for fold, (train_idx, test_idx) in enumerate(tscv.split(weekly)):
    print(f"Fold {fold}: train rows {len(train_idx)}, test rows {len(test_idx)}")
```

GEMINI ASSISTANT PROMPT

“Act as a Python time-series analyst. I have a pandas DataFrame `weekly` sorted by date with a date column. Help me create a chronological train/validation/test split and explain why random splitting is inappropriate for this dataset. Then show me how to use scikit-learn’s `TimeSeriesSplit` for walk-forward validation. Generate executable code and explain each step.”

What to check: confirm the code never shuffles the data and that any feature built with `.rolling()` or `.shift()` is computed *before* the split, using only past values relative to each row.

KEY TAKEAWAY

Getting the split wrong invalidates every evaluation metric downstream — this is the single most common error in applied forecasting work.

MODULE 10 — Feature Engineering

Figure 4 — the weekly case series branches into three feature families, which converge into the modelling dataset.

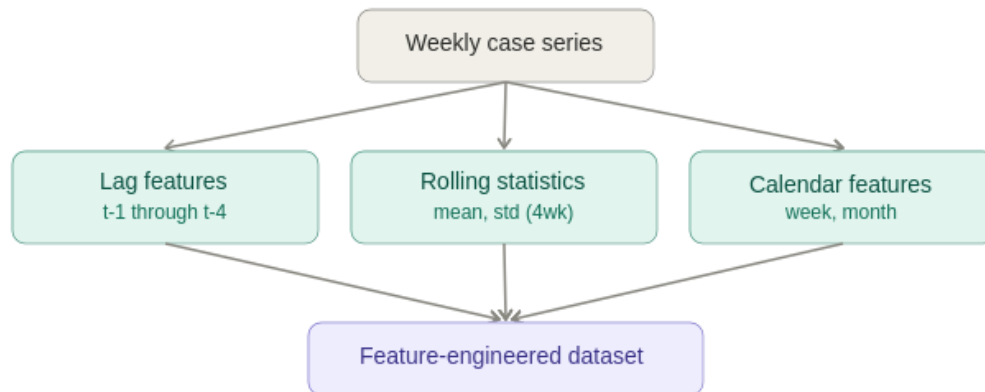


Figure 4: Feature engineering pipeline

```
fe = weekly.copy()
for lag in [1, 2, 3, 4]:
    fe[f"lag_{lag}"] = fe["weekly_cases"].shift(lag)

fe["rolling_mean_4"] = fe["weekly_cases"].shift(1).rolling(4).mean()
fe["rolling_std_4"] = fe["weekly_cases"].shift(1).rolling(4).std()
fe["week_of_year"] = fe["date"].dt.isocalendar().week.astype(int)
fe["month"] = fe["date"].dt.month

fe = fe.dropna().reset_index(drop=True)
```

CODE WALKTHROUGH

- Lag features (lag_1...lag_4) let a non-time-series model (e.g., Random Forest) use recent history as ordinary predictors.
- Rolling statistics are computed on `.shift(1)` — i.e., **excluding** the current week — to avoid leakage.
- Calendar features (week_of_year, month) let tree-based models learn seasonality without an explicit seasonal decomposition.

GEMINI ASSISTANT PROMPT

“I have a weekly disease-cases DataFrame called `fe` with columns `date` and `weekly_cases`. Generate Python code to create lag features for lags 1 through 4, a 4-week rolling mean and standard deviation computed only from past values (no leakage), and calendar features for week-of-year and month. Explain why each rolling feature must be shifted before being computed.”

Common Gemini mistake: computing `rolling(4).mean()` without a preceding `.shift(1)`, which includes the current week's own value in its own predictor — leakage.
How to validate: for any row, `rolling_mean_4` should be computable from strictly earlier rows than the target row.

KEY TAKEAWAY

Good features encode epidemiological reasoning (recent momentum, seasonality) into numbers a model can use — and every one must respect the boundary between “known at forecast time” and “not yet known.”

MODULE 11 — Model Development

Model	When to use	Strengths	Weaknesses
Naive / seasonal naive	Always, as baseline	Transparent, zero training	Captures no dynamics
Linear / Ridge regression	Simple, interpretable trend + lag relationships	Fast, interpretable coefficients	Misses non-linear patterns
Random Forest / Gradient Boosting	Non-linear relationships among lag/calendar features	Handles non-linearity, feature importance available	Less interpretable, can overfit with little data
SARIMA	Classical time-series with clear seasonality	Statistically principled, native prediction intervals	Requires stationarity work, slower to tune
LSTM / advanced neural nets (optional)	Large datasets, complex multivariate dependencies	Can model complex sequences	Needs lots of data, hard to interpret, easy to overfit — not required for most surveillance teams

Do not reach for deep learning by default. With a few hundred weekly observations for one location, a well-engineered Random Forest or SARIMA model will usually outperform an LSTM and remain explainable to non-technical stakeholders.

GEMINI ASSISTANT PROMPT

“I’m forecasting weekly disease case counts with about 250 historical weekly observations for one location. Compare Random Forest regression and SARIMA for this use case in terms of data requirements, interpretability, and suitability for public-health reporting. Do not recommend deep learning unless you can justify it for a dataset this size.”

What to check: push back if Gemini defaults to recommending a neural network “for best accuracy” — with this data volume, that recommendation is not epidemiologically or statistically justified, and Gemini should say so if prompted correctly.

MODULE 12 — Model Training and Troubleshooting

Model A: Random Forest on engineered features

```
from sklearn.ensemble import RandomForestRegressor

feature_cols = ["lag_1", "lag_2", "lag_3", "lag_4", "rolling_mean_4",
               "rolling_std_4", "week_of_year", "month"]

X_train, y_train = train_fe[feature_cols], train_fe["weekly_cases"]
X_test, y_test = test_fe[feature_cols], test_fe["weekly_cases"]

rf = RandomForestRegressor(n_estimators=300, random_state=42)
rf.fit(X_train, y_train)
rf_preds = rf.predict(X_test)
```

Model B: SARIMA

```
from statsmodels.tsa.statespace.sarimax import SARIMAX

sarima_model = SARIMAX(
    train["weekly_cases"],
    order=(1, 1, 1),
    seasonal_order=(1, 1, 1, 52),
    enforce_stationarity=False,
    enforce_invertibility=False,
)
sarima_fit = sarima_model.fit(dispatch=False)
sarima_forecast = sarima_fit.get_forecast(steps=len(test))
sarima_preds = sarima_forecast.predicted_mean
sarima_ci = sarima_forecast.conf_int()
```

GEMINI ASSISTANT PROMPT

“Here is my SARIMAX fitting code [paste code]. Explain what `order=(1,1,1)` and `seasonal_order=(1,1,1,52)` mean in plain language, what `get_forecast` returns, and what common convergence warnings mean and how to address them. Do not change my model specification — just explain and help me debug.”

How to verify Gemini’s code: re-run any Gemini-suggested fix on your validation set only, never your held-out test set, and confirm the output shapes and index alignment (`sarima_preds.index` must match `test.index`) before trusting numbers.

GEMINI CHALLENGE

Ask Gemini to explain a `ConvergenceWarning` from `sarima_fit.fit()`. Then independently verify its explanation against the statsmodels documentation before acting on it.

MODULE 13 — Model Evaluation

```
from sklearn.metrics import mean_absolute_error, mean_squared_error

def evaluate(y_true, y_pred, label):
    mae = mean_absolute_error(y_true, y_pred)
    rmse = mean_squared_error(y_true, y_pred, squared=False)
    smape = 100 * np.mean(2 * np.abs(y_pred - y_true) / (np.abs(y_true) +
np.abs(y_pred) + 1e-9))
    print(f"{label:15s} MAE={mae:8.2f} RMSE={rmse:8.2f}
sMAPE={smape:6.2f}%")

evaluate(y_test, baseline_df.loc[y_test.index, "naive"], "Naive baseline")
evaluate(y_test, rf_preds, "Random Forest")
evaluate(test["weekly_cases"], sarima_preds, "SARIMA")
```

Why sMAPE instead of MAPE: MAPE divides by the actual value, so it becomes unstable or undefined when weekly cases are near zero (common during low-transmission periods) — sMAPE’s symmetric denominator handles that far more gracefully. Report R^2 only when the relationship is meaningfully linear; forecasting error is usually better communicated through MAE/RMSE in the original case-count units, which are directly interpretable by public-health stakeholders.

GEMINI ASSISTANT PROMPT

“I’m evaluating disease-forecasting models and considering MAPE. Explain why MAPE is problematic when actual case counts are near zero, and suggest an alternative metric with the formula. Do not just say ‘use RMSE instead’ — explain the tradeoffs.”

KEY TAKEAWAY

Never report a single metric in isolation. Compare every candidate model against the Module 8 baseline, on the same held-out test set, using a metric that behaves sensibly at your data’s actual scale.

MODULE 14 — Uncertainty and Forecast Intervals

A forecast is never a single number. Report it as a range.

```
# SARIMA gives native prediction intervals
print(sarima_ci.tail()) # Lower / upper bounds per forecasted week

# For Random Forest, approximate intervals from the spread across trees
tree_preds = np.stack([tree.predict(X_test) for tree in rf.estimators_])
lower = np.percentile(tree_preds, 5, axis=0)
upper = np.percentile(tree_preds, 95, axis=0)
```

What a 90% prediction interval means: if we repeated this forecasting process many times under similar conditions, about 90% of intervals would contain the true value. It does **not** mean there's a 90% chance next week's actual count falls in this specific interval, and it does not account for structural changes (e.g., a new intervention) that weren't in the training history.

KEY TAKEAWAY

Always report “127 cases (90% interval: 105–151),” never “127 cases.” The interval communicates honestly what the model does and doesn't know.

MODULE 15 — Outbreak Detection and Early Warning

Forecasting (what's likely next) is distinct from outbreak detection (is what we're seeing unusual enough to act on).

Historical baseline → Expected cases → Observed cases → Deviation from expected → Alert threshold → Potential outbreak signal

Figure 5 — an alert is the output of this chain, not a confirmed outbreak; investigation happens after the signal, not instead of it.



Figure 5: Outbreak early warning workflow

```
baseline_mean = train["weekly_cases"].rolling(8).mean()
baseline_std = train["weekly_cases"].rolling(8).std()
```

```
z_score = (weekly["weekly_cases"] - baseline_mean.reindex(weekly.index,
method="ffill")) / \
    baseline_std.reindex(weekly.index, method="ffill")

weekly["alert"] = z_score > 2    # flag weeks more than 2 standard deviations
above baseline
```

An alert is a prompt to investigate — not a confirmed outbreak. Confirmation requires epidemiological investigation (lab confirmation, field verification, ruling out reporting artifacts), which a threshold rule cannot do on its own.

GEMINI CHALLENGE

Ask Gemini to suggest a control-chart-based alternative to the z-score threshold above (e.g., a CUSUM approach). Independently verify the formula it gives against a statistics reference before using it — do not paste it into a live surveillance workflow untested.

KEY TAKEAWAY

A statistical alert is the start of an investigation, not the end of one.

MODULE 16 — Interpreting the Model

```
import pandas as pd
importances = pd.Series(rf.feature_importances_,
index=feature_cols).sort_values(ascending=False)
print(importances)
```

Feature importance does not imply causation. If lag_1 dominates, that tells you recent case counts are the strongest statistical predictor of next week's count — not that last week's cases *caused* this week's. Confounding, shared seasonal drivers, and reporting cycles can all produce strong lag relationships without any direct causal mechanism beyond the epidemiological transmission process itself.

REFLECTION QUESTIONS

1. If week_of_year ranks highly, what does that suggest — and what would you need to check before calling it “seasonality” rather than a reporting-calendar artifact?
 2. When should this model *not* be trusted? (Hint: think about structural breaks — new variants, new interventions, population changes — not represented in its training history.)
-

MODULE 17 — Gemini as an AI Assistant: Scope and Limits

Gemini CAN help with

Explaining Python and statistical concepts; generating boilerplate code; debugging error messages; suggesting visualizations; documenting code; drafting test cases; improving plot readability; drafting report structure.

Gemini must NEVER be the one to decide

- Whether a disease outbreak actually exists
- Whether a data point is an error or a genuine signal, and should be removed
- Whether a variable is epidemiologically valid to include
- Whether a correlation implies causation
- Whether a model is scientifically appropriate to publish or act on
- Whether a public-health intervention should be implemented

These are judgment calls that require epidemiological training, context the AI does not have, and accountability a chat assistant cannot carry.

Two-Column Comparison

Traditional workflow	AI-assisted workflow
You write the pandas/statsmodels code	You ask Gemini to draft or explain code
You understand the algorithm’s assumptions	Gemini explains the algorithm in plain language
You debug tracebacks by reading them	Gemini suggests likely causes and fixes
You interpret the output against domain knowledge	Gemini offers a second, independently-checked interpretation
You validate results against the traditional run	You never accept Gemini’s output unverified

KEY TAKEAWAY

Gemini is a tutor, coding partner, and debugging assistant — never the analyst of record. You must be able to explain and defend the final model without it.

MODULE 18 — Prompt Engineering for Modelling Tasks

Structure: ROLE + CONTEXT + DATA + TASK + CONSTRAINTS + OUTPUT FORMAT + VALIDATION

Bad prompt: “Build me a forecasting model.”

Better prompt: “Act as a Python time-series analyst. I have a pandas DataFrame df with columns week (date) and cases (int). Help me create a chronological train/test split and explain why random splitting is inappropriate here. Generate executable Python code and explain each step.”

Expert-level prompt: “Act as a Python time-series analyst reviewing code for a public-health surveillance team. I have a weekly cases DataFrame with lag and rolling-mean features already engineered [schema below]. Generate a Random Forest training script using scikit-learn, with a fixed random_state for reproducibility, that trains only on data before 2023-01-01. After training, print feature importances and a table comparing MAE/RMSE against a naive baseline computed on the same test period. Flag anywhere the code could leak future information into a feature, and explain how you checked for that.”

Don't Trust AI Blindly — Audit Checklist

Before running any Gemini-generated code, check: imports and library names are real; variable names match your actual DataFrame; train/test split is chronological, not random; no .rolling() or .shift() computes on the current/future row; metrics are computed only on the held-out test set; no deprecated function calls; no unverifiable claims about “expected accuracy” stated as fact.

GEMINI CHALLENGE

Below is deliberately flawed code. Identify the bug before reading the explanation.

```
# FLAWED – find the bug
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
shuffle=True)
```

Bug: shuffle=True (the default) randomly splits a time series, causing temporal leakage — the model can train on data from after the test period. It should use a chronological split (Module 9) instead.

MODULE 19 — Data Privacy, Ethics, and Responsible AI Use

Public health data ranges from fully public aggregate counts (like the OWID dataset used in this workbook) to highly sensitive, identifiable patient records. The rules change accordingly.

Before sending any data to Gemini, ask: - Is this data public, or could it identify an individual? - Has it been properly anonymized or aggregated (not just had names removed — small-cell counts in a rural district can still be re-identifying)? - Does my organization's data-governance policy permit sharing this externally? - Can I complete the task using a synthetic sample or a public dataset instead?

Rule for this workbook and beyond: never paste patient-identifiable information into a public AI chat tool. Use synthetic, aggregated, or public datasets when practicing or prototyping, and follow your institution's approved, governed AI tools for any real identifiable surveillance data.

KEY TAKEAWAY

Responsible AI use in public health starts before you open the chat window — with a clear-eyed check of what you're about to share.

MODULE 20 — Capstone Project

Figure 6 — the whole workbook condensed into one end-to-end architecture: this is what your capstone should be able to walk a colleague through, phase by phase.

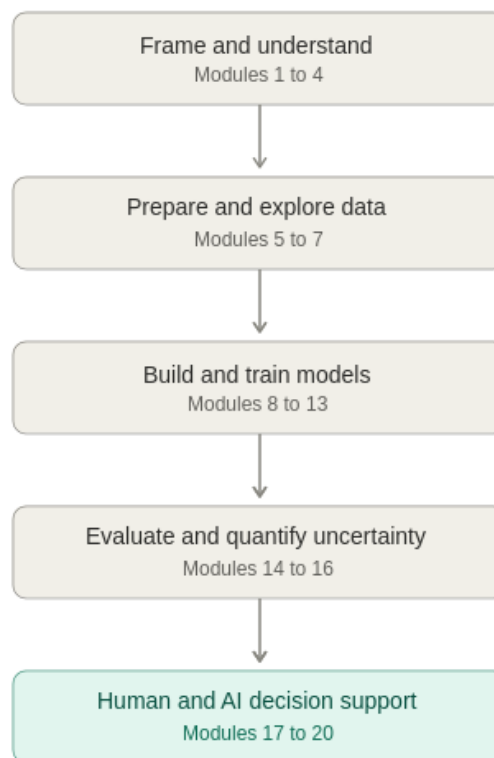


Figure 6: End to end workbook architecture

Task: "Develop a 4-week disease-incidence forecasting model using historical surveillance data, and evaluate whether it can provide useful early-warning information."

Independently complete, using your own surveillance data or a public dataset (OWID, WHO GHO, or a national dataset):

1. Problem statement (Module 2 format)
2. Data ingestion and quality inspection
3. Cleaning, with documented justification for each decision
4. EDA with at least two interpreted visualizations
5. Time-series diagnostics (trend, seasonality, stationarity)
6. Baseline model
7. Chronologically split feature-engineered dataset
8. At least two forecasting models (e.g., Random Forest and SARIMA)
9. Evaluation against the baseline using MAE/RMSE/sMAPE
10. A forecast with an uncertainty interval
11. A simple early-warning threshold applied to the forecast
12. Feature importance / interpretation, with explicit causation caveats
13. A one-page technical summary: method, results, and — critically — limitations
14. A completed AI-Assistance Log (Appendix A)

APPENDIX A — AI-Assistance Log Template

Keep this log throughout your project — it is part of your capstone deliverable and supports reproducibility and transparency about where AI assistance was used.

Date	Task	Gemini Prompt (summary)	Response Used?	What Was Changed?	How Was It Validated?

APPENDIX B — Troubleshooting Guide

Error	Likely cause	Traditional fix	Gemini prompt	Verification
FileNotFoundError	Wrong path, or file not re-uploaded after runtime reset	Re-upload / re-mount Drive; print <code>os.listdir()</code> to confirm path	“Here is my file path and error — what’s wrong?”	Confirm the file actually appears in the printed directory listing

Error	Likely cause	Traditional fix	Gemini prompt	Verification
KeyError: 'column_name'	Column name typo or column dropped earlier	<code>print(df.columns)</code> to check exact names	"My DataFrame has these columns [list] — why does <code>df['x']</code> fail?"	Match the name exactly, including case and whitespace
ValueError: could not convert string to float	Non-numeric text (e.g., "N/A") in a numeric column	<code>pd.to_numeric(col, errors='coerce')</code> then re-check NaNs	"This column has mixed types — how do I coerce it safely and see what became NaN?"	Compare row count before/after; inspect what turned to NaN
Date parsing errors	Inconsistent date formats	<code>pd.to_datetime(col, format=..., errors='coerce')</code>	"My dates are in mixed formats — how do I parse them safely?"	Check for unexpected NaT values after parsing
Convergence Warning (SARIMA)	Poor initial parameters, non-stationary series	Re-check differencing order; try <code>enforce_stationarity=False</code>	"Explain this SARIMAX convergence warning and how to address it"	Re-fit on validation set only; compare AIC before/after
Train/test shape mismatch	Feature engineering dropped differently	Recompute features before splitting, then split	"Why do <code>X_train</code> and <code>X_test</code> have different columns/r	<code>assert X_train.columns.equals(X_test.columns)</code>

Error	Likely cause	Traditional fix	Gemini prompt	Verification
	t rows in train vs. test		ows?"	
Insufficient data for seasonal model	Fewer than ~2 full seasonal cycles	Use a simpler model or aggregate to a coarser resolution	"Is my dataset long enough for seasonal_order period=52?"	Count distinct years in your training data

APPENDIX C — Glossary

Epidemiology — the study of how disease is distributed and what determines that distribution in populations. **Surveillance** — the ongoing, systematic collection and analysis of health data. **Outbreak / Epidemic / Pandemic** — a localized, regional, and global rise in disease activity above expectation, respectively. **Incidence** — the number of new cases in a period. **Prevalence** — the total number of existing cases at a point in time. **Time series** — data points ordered by time. **Trend** — the long-run direction. **Seasonality** — a pattern that repeats at fixed intervals. **Stationarity** — a series whose statistical properties don't change over time. **Autocorrelation** — the correlation of a series with its own past values. **Lag feature** — a past value of a series used as a predictor for a future value. **Baseline** — a simple reference forecast (e.g., naive) that any real model must outperform. **Train / validation / test set** — chronologically ordered data subsets used to fit, tune, and finally evaluate a model. **Data leakage** — when information from outside the training period improperly influences the model. **Overfitting / Underfitting** — a model that memorizes noise / a model too simple to capture real patterns. **MAE / RMSE / MAPE / sMAPE** — Mean Absolute Error, Root Mean Squared Error, Mean/Symmetric Mean Absolute Percentage Error — forecast accuracy metrics. **Prediction interval** — a range expected to contain a future observation with a stated probability, under the model's assumptions. **Feature importance** — a measure of how much a model relies on a given input; not evidence of causation. **Early warning** — a statistical alert that observed activity deviates from an expected baseline, prompting investigation.

APPENDIX D — References and Data Sources

- Our World in Data, COVID-19 dataset — <https://covid.ourworldindata.org/data/owid-covid-data.csv> (documentation: <https://github.com/owid/covid-19-data>)
- World Health Organization, Global Health Observatory — <https://www.who.int/data/gho>
- WHO Disease Outbreak News — <https://www.who.int/emergencies/disease-outbreak-news>
- CDC Data & Statistics — <https://www.cdc.gov/datastatistics/>
- pandas documentation — <https://pandas.pydata.org/docs/>
- scikit-learn documentation — <https://scikit-learn.org/stable/>
- statsmodels documentation — <https://www.statsmodels.org/stable/index.html>
- Google Colab documentation — <https://colab.research.google.com/notebooks/intro.ipynb>
- Google Gemini documentation — <https://ai.google.dev/gemini-api/docs>

Note: verify all dataset and documentation links against their live source before each training delivery, as URLs and dataset schemas may change over time.